

Integration Review:

X12Bridge -> JSON Claims Integrity

Date: March 4, 2026

Status: Proposed Architecture -- Deeper Integration

The Core Challenge

X12Bridge outputs flat segment arrays -- each segment as {segment_id, elements[], raw} -- while the Haskell DSL engine expects semantic claim JSON with fields like claim.amount, provider.npi, patient.name.first, etc. The SegmentMapper (Step 2 in the plan) bridges this gap.

What's Sound in the Current Plan

1. Mix path dependency -- confirmed working (../X12Bridge from phoenix_web/), only adds phoenix_ecto transitively
2. The segment-to-field mapping table -- correctly maps X12 element positions to the Haskell engine's expected fields
3. Scoping to 837P first -- smart; 837I/D share ~80% of segments and can be added later
4. Leveraging identify_loops/1 -- X12Bridge already groups segments by CLM + LX, so you don't need to write loop detection

Areas Worth Discussing

Topic	Consideration
Database overlap	X12Bridge has its own conversion_batches + conversion_jobs tables. Do you want to reuse those (track X12 provenance) or keep the existing batches + edi_files tables as the single store?
Two-stage pipeline	X12Bridge separates verify (free) from translate (billed). The plan calls Verifier.verify then Converter.convert_content in sequence. Should the UI expose the two stages separately, or just run both transparently on upload?
Round-trip validation	X12Bridge has RoundtripValidator that ensures JSON->X12 reconstruction matches the original byte-for-byte. The plan doesn't mention it -- worth including as a data integrity gate.
Remote import	X12Bridge already supports HTTP/SFTP/Databricks/local path fetching. This could be exposed in the claims integrity UI for pulling X12 files from external systems without manual download.
Multi-claim files	Already handled. X12Bridge's ClaimSplitter splits multi-claim X12 files into individual claims, expands them into separate database jobs, and translates each independently. By the time claims reach this app via the webhook, each is already a standalone JSON object -- no additional splitting needed on this side.

Recommended Approach

The plan as written is implementable. The main decision point is:

Thin integration (the current plan)

Just add the mapper, keep X12Bridge as a library, no DB sharing. Simplest path.

Deeper integration

Also bring in X12Bridge's batch tracking, remote import, and round-trip validation. More capability but more surface area.

Deeper Integration: Approach Selected

Rewriting X12Bridge in Haskell -- Don't Do This

Agreed, this is a bad idea. X12Bridge is ~2,500+ lines of battle-tested Elixir with round-trip validation, claim splitting, remote fetching, and a two-stage pipeline. Translating it to Haskell would be:

- Months of work for zero new capability
- Two parsers to maintain instead of one
- Loss of the LiveView UI you already have for batch monitoring

The X12 parsing problem is well-solved in Elixir. The Haskell engine's strength is DSL rule evaluation -- keep each tool doing what it does best.

How to Pass Claims Between the Two Apps

Method	Pros	Cons
ZIP on disk	Simple, auditable, works offline	Polling needed, stale file risk, no delivery guarantee
Shared DB table	Atomic, queryable, both use PG	Tight coupling, schema coordination, migration headaches
Message queue / HTTP push	Decoupled, real-time, retry built-in	Extra infrastructure (unless HTTP)

Recommended: HTTP Push (webhook-style)

The simplest "deeper integration" that avoids shared state:

1. X12Bridge runs on its cron schedule -- fetches X12 batches from SFTP/HTTP/Databricks (it already does this)
2. After translation, X12Bridge POSTs the JSON claims to a new endpoint on this app: POST /api/x12-batch-ingest
3. This app receives the batch, stores it in batches + edi_files, and it's immediately available for rule evaluation
4. No shared database, no polling, no ZIP files

X12Bridge already has RemoteFetcher for scheduled imports and Conversions for batch orchestration. Adding an HTTP POST after successful translation is a small addition -- maybe 50 lines of Elixir.

This also means X12Bridge can run on a different machine entirely if needed later.

What the Cron Flow Would Look Like



SegmentMapper Placement

The SegmentMapper should live in X12Bridge -- that way this app never needs to know about X12 segment structure, and X12Bridge owns the full translation pipeline end-to-end.